WOXSEN UNIVERSITY, HYDERABAD

DBMS COURSE PROJECT | PROJECT NO. 67

Design and Implementation of a Database Management System for Gym Membership and Fitness Session Management System

Project Report

Name	Roll number
Sai Nihal Nellore	25WU0101118
[Member 2 name]	*[Roll number]*
[Member 3 name]	*[Roll number]*

Course	Database Management Systems (DBMS)
Section	*[Section name: AIML Wolves / CSE Tigers / CSE Rhinos]*
Faculty	*[Faculty name]*
Academic year	2026-27

Contents

1. Cover page
2. Abstract
3. Introduction and problem statement
4. Objectives and scope
5. Software and hardware requirements
6. ER diagram
7. Relational schema and normalization
8. Data dictionary
9. SQL commands used (DDL, DML) with sample outputs
10. Queries with outputs
11. UI design and screenshots
12. Implementation details
13. Testing
14. Conclusion and future enhancements
15. References
16. Contribution of each member
17. Appendix: GitHub repository link

2. Abstract

This project designs and implements a relational database for a gym that sells membership plans and runs group fitness sessions. The database, built in MySQL, has seven tables: MEMBERSHIP_PLAN, MEMBER, TRAINER, MEMBERSHIP, PAYMENT, FITNESS_SESSION and BOOKING. The design started from an entity-relationship diagram, was mapped to a relational schema and normalized to third normal form.

Data integrity is enforced inside the database through 7 primary keys, 6 foreign keys, 7 UNIQUE constraints and 12 CHECK constraints, with ON DELETE CASCADE and ON DELETE RESTRICT chosen per relationship. The sample data set holds 59 records, with at least five in every table. Fifteen SELECT queries, including multi-table joins, aggregate functions, GROUP BY with HAVING and subqueries, answer practical questions such as revenue per plan, seats left per session and fees still due.

A web interface written in Python with Flask connects to the database through the PyMySQL driver. It lets staff view, insert and delete members, trainers, memberships, sessions and bookings, and every change is immediately visible in MySQL.

3. Introduction and problem statement

3.1 Introduction

A gym deals with three kinds of information every day: who its members are and what they have paid for, which trainer is running which session, and which members have booked a seat in those sessions. These facts are closely related. A booking is only meaningful if the member and the session exist, and a payment is only meaningful against a particular membership.

3.2 Problem statement

In many small and medium gyms this information is kept in paper registers or in separate spreadsheets for fees, class lists and attendance. This causes the following problems:

- **Duplicate and inconsistent data.** A member's name and phone number are written again in every register, so a change in one place is missed in the others.
- **No tracking of renewals and dues.** Staff cannot easily list the memberships that are about to expire or the fees that are only part paid.
- **Overbooked sessions.** Nothing checks the capacity of a session or stops the same member being entered twice.
- **No reports.** Questions such as "which plan earns the most" or "which trainer handles the most bookings" need manual counting.

The aim of this project is to replace these registers with one relational database, in which each fact is stored once, the rules are enforced by the DBMS, and the questions above are answered by SQL queries.

3.3 Intended users

User	What they use the system for
Front desk staff	Register members, sell plans, record payments, take and cancel bookings
Gym manager	Revenue, plan, trainer workload and attendance reports
Trainers	Session timetable and the list of members booked
Members (indirect)	Correct plan dates, payment records and a confirmed seat

4. Objectives and scope

4.1 Objectives

1. Store members, plans, trainers, sessions, bookings and payments in a single MySQL database.
2. Design the database from an ER diagram and normalize the schema to third normal form.
3. Enforce integrity with primary keys, foreign keys, UNIQUE, CHECK, NOT NULL and DEFAULT constraints.
4. Write SQL queries, including joins and aggregate functions, that answer day-to-day questions.
5. Build a user interface connected to the database that supports insertion, deletion and viewing of records.

4.2 Scope

In scope	Out of scope
Member registration Membership plans and plan purchase Fee payments, including part payments Trainer details and session scheduling Session booking and attendance status Reports on revenue, workload and attendance	Online payment gateway User login and role-based access Equipment and inventory management Mobile application for members Staff payroll

5. Software and hardware requirements

5.1 Software

Component	Requirement
Operating system	Windows 10/11, macOS or Linux
Database	MySQL Server 8.0 (the script also runs on MariaDB 10.4 or later)
Database client	MySQL Command Line Client or MySQL Workbench
Programming language	Python 3.9 or later
Web framework	Flask 3.x
Database driver	PyMySQL 1.1
Front end	HTML5, CSS3, Jinja2 templates
Browser	Chrome, Edge or Firefox
Version control	Git and GitHub

5.2 Hardware

Component	Minimum
Processor	Dual-core, 1.6 GHz or faster
RAM	4 GB (8 GB recommended)
Disk space	About 1 GB for MySQL, Python and the project
Display	1366 × 768 or higher

6. ER diagram

The diagram uses Chen notation: rectangles are entities, diamonds are relationships, ovals are attributes, and underlined attributes are primary keys. A double line shows total participation.

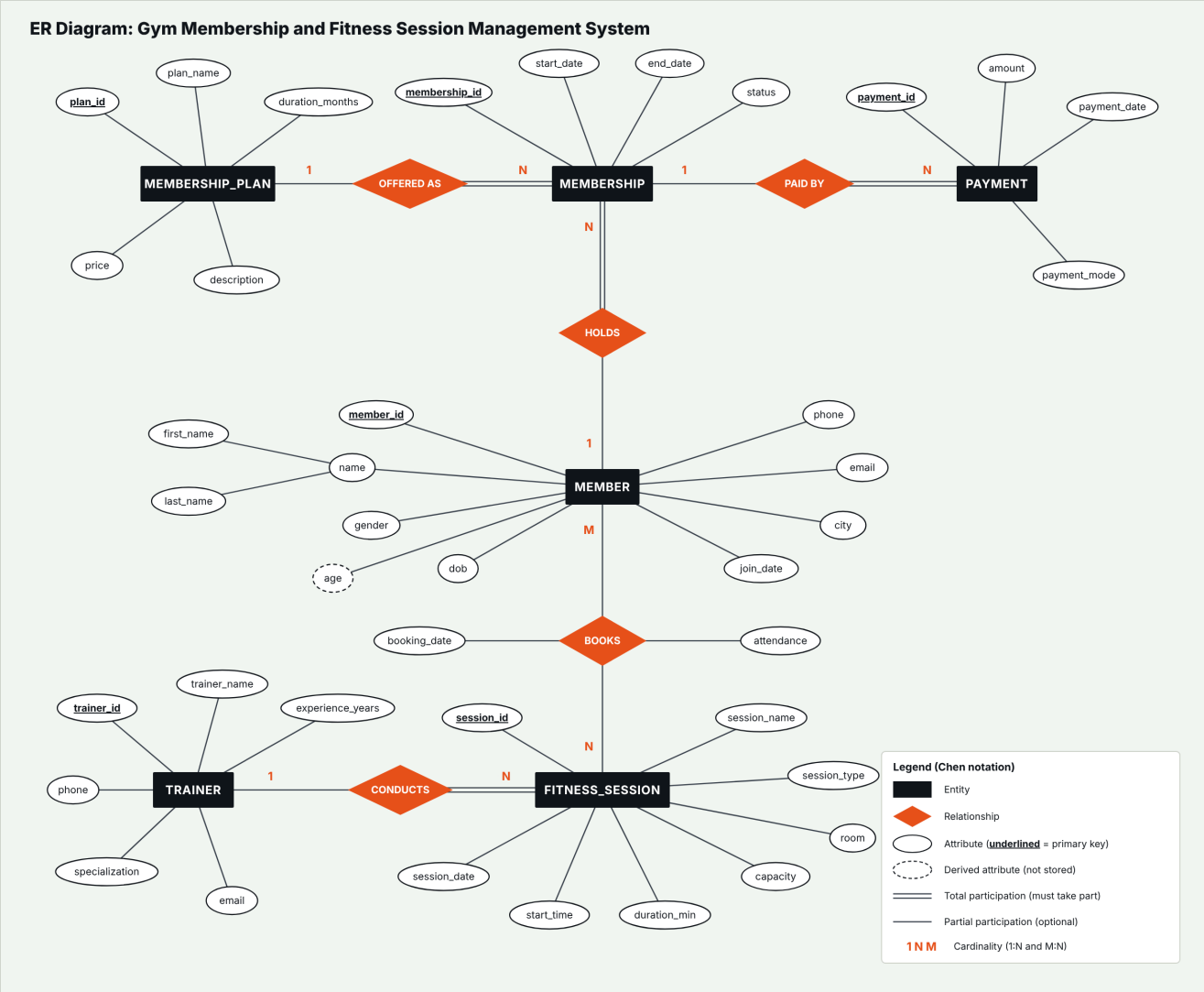


Figure 6.1: ER diagram of the Gym Membership and Fitness Session Management System

6.1 Entities

Entity	Primary key	Other attributes
MEMBERSHIP_PLAN	plan_id	plan_name, duration_months, price, description
MEMBER	member_id	name (composite: first_name, last_name), gender, dob, age (derived), phone, email, city, join_date
TRAINER	trainer_id	trainer_name, specialization, phone, email, experience_years
MEMBERSHIP	membership_id	start_date, end_date, status
PAYMENT	payment_id	amount, payment_date, payment_mode
FITNESS_SESSION	session_id	session_name, session_type, session_date, start_time, duration_min, capacity, room

6.2 Relationships, cardinality and participation

Relationship	Between	Cardinality	Participation	Meaning
HOLDS	MEMBER and MEMBERSHIP	1 : N	MEMBER partial, MEMBERSHIP total	A member can buy or renew plans many times; each membership belongs to one member.
OFFERED AS	MEMBERSHIP_PLAN and MEMBERSHIP	1 : N	PLAN partial, MEMBERSHIP total	Each membership is of exactly one plan; a plan may not have been sold yet.
PAID BY	MEMBERSHIP and PAYMENT	1 : N	MEMBERSHIP partial, PAYMENT total	A membership can be paid in instalments; each payment is for one membership.
CONDUCTS	TRAINER and FITNESS_SESSION	1 : N	TRAINER partial, SESSION total	Every session has exactly one trainer; a trainer may have no session.
BOOKS	MEMBER and FITNESS_SESSION	M : N	Both partial	A member books many sessions and a session is booked by many members. Attributes: booking_date, attendance.

7. Relational schema and normalization

7.1 Mapping the ER diagram to tables

- Each of the six entities becomes a table with the same primary key.
- Each 1:N relationship is implemented as a foreign key on the N side (for example `trainer_id` in FITNESS_SESSION).
- The M:N relationship BOOKS becomes the table BOOKING, holding the two foreign keys and the relationship's own attributes. A surrogate key `booking_id` is used as primary key and the pair (member_id, session_id) is declared UNIQUE.
- The composite attribute name is stored as its parts first_name and last_name. The derived attribute age is not stored; it is calculated from dob in queries.

7.2 Relational schema

MEMBERSHIP_PLAN (plan_id, plan_name, duration_months, price, description)
MEMBER (member_id, first_name, last_name, gender, dob, phone, email, city, join_date)
TRAINER (trainer_id, trainer_name, specialization, phone, email, experience_years)
MEMBERSHIP (membership_id, *member_id*, *plan_id*, start_date, end_date, status)
PAYMENT (payment_id, *membership_id*, amount, payment_date, payment_mode)
FITNESS_SESSION (session_id, session_name, session_type, *trainer_id*, session_date, start_time, duration_min, capacity, room)
BOOKING (booking_id, *member_id*, *session_id*, booking_date, attendance)

Underlined = primary key. Italic = foreign key.

Foreign key	References	On delete	Reason
membership.member_id	member.member_id	CASCADE	A membership has no meaning without its member.
membership.plan_id	membership_plan.plan_id	RESTRICT	A plan that has been sold must not be deleted.
payment.membership_id	membership.membership_id	CASCADE	Payments belong to their membership.
fitness_session.trainer_id	trainer.trainer_id	RESTRICT	A trainer with scheduled sessions must not be deleted.
booking.member_id	member.member_id	CASCADE	Bookings of a removed member are removed.
booking.session_id	fitness_session.session_id	CASCADE	Bookings of a cancelled session are removed.

7.3 Normalization

Unnormalized form. A gym register kept as one table would look like this:

```
GYM_REGISTER (member_name, phone, plan_name, plan_price, plan_months, start_date, fees_paid {amount, date, mode}, sessions_booked {session_name, date, trainer_name, trainer_phone})
```

It has repeating groups (several payments and several sessions in one row) and repeats plan and trainer details for every member.

First normal form (1NF). Every attribute must be atomic and there must be no repeating groups. The repeating groups are removed so that there is one row per payment and one row per booking, and member_name is split into first_name and last_name. Identifiers member_id, plan_id, session_id and trainer_id are introduced.

Second normal form (2NF). The table must be in 1NF and no non-key attribute may depend on only part of a composite key. In the booking data the key is (member_id, session_id). The functional dependencies are:

- member_id → first_name, last_name, gender, dob, phone, email, city, join_date
- session_id → session_name, session_type, session_date, start_time, duration_min, capacity, room, trainer_id
- (member_id, session_id) → booking_date, attendance

The first two are partial dependencies, so the data is split into MEMBER, FITNESS_SESSION and BOOKING.

Third normal form (3NF). The table must be in 2NF and no non-key attribute may depend on another non-key attribute. The transitive dependencies found were:

- session_id → trainer_id → trainer_name, specialization, phone, email. Trainer details move to TRAINER.
- membership_id → plan_id → plan_name, duration_months, price. Plan details move to MEMBERSHIP_PLAN.
- payment_id → membership_id → member_id, plan_id, start_date, end_date. Membership details move to MEMBERSHIP, leaving PAYMENT with only its own facts.

In the final seven tables every non-key attribute depends on the primary key of its own table and on nothing else, so the schema is in 3NF. Two design choices are worth noting. First, `end_date` is stored in MEMBERSHIP even though it can be estimated from start_date and the plan's duration, because it records the date actually agreed with the member and must not change if the plan's duration is edited later. Second, the amount in PAYMENT is stored separately from the plan price because a membership can be paid in instalments.

8. Data dictionary

8.1 MEMBERSHIP_PLAN

Plans the gym sells.

Column	Data type	Constraints	Meaning
plan_id	INT(11)	PRIMARY KEY, AUTO_INCREMENT	Plan number
plan_name	VARCHAR(50)	UNIQUE; NOT NULL	Name of the plan
duration_months	INT(11)	NOT NULL; CHECK between 1 and 24	Length of the plan in months
price	DECIMAL(8,2)	NOT NULL; CHECK price > 0	Plan fee in rupees
description	VARCHAR(150)		What the plan includes

8.2 MEMBER

People registered at the gym.

Column	Data type	Constraints	Meaning
member_id	INT(11)	PRIMARY KEY, AUTO_INCREMENT	Member number
first_name	VARCHAR(30)	NOT NULL	First name
last_name	VARCHAR(30)	NOT NULL	Last name
gender	CHAR(1)	NOT NULL; CHECK in ('M','F','O')	M, F or O
dob	DATE	NOT NULL	Date of birth
phone	CHAR(10)	UNIQUE; NOT NULL	10-digit mobile number
email	VARCHAR(60)	UNIQUE; NOT NULL	Email address
city	VARCHAR(40)	NOT NULL; DEFAULT 'Hyderabad'	City of residence
join_date	DATE	NOT NULL	Date of registration

8.3 TRAINER

Staff who conduct sessions.

Column	Data type	Constraints	Meaning
trainer_id	INT(11)	PRIMARY KEY, AUTO_INCREMENT	Trainer number
trainer_name	VARCHAR(60)	NOT NULL	Full name
specialization	VARCHAR(30)	NOT NULL	Main training area
phone	CHAR(10)	UNIQUE; NOT NULL	10-digit mobile number
email	VARCHAR(60)	UNIQUE; NOT NULL	Email address
experience_years	INT(11)	NOT NULL; DEFAULT 0; CHECK >= 0	Years of experience

8.4 MEMBERSHIP

One row each time a member buys or renews a plan.

Column	Data type	Constraints	Meaning
membership_id	INT(11)	PRIMARY KEY, AUTO_INCREMENT	Membership number
member_id	INT(11)	FOREIGN KEY → member, ON DELETE CASCADE; NOT NULL	Member number
plan_id	INT(11)	FOREIGN KEY → membership_plan, ON DELETE RESTRICT; NOT NULL	Plan number
start_date	DATE	NOT NULL	First day of the plan
end_date	DATE	NOT NULL; CHECK end_date > start_date	Last day of the plan
status	VARCHAR(10)	NOT NULL; DEFAULT 'Active'; CHECK in ('Active','Expired','Cancelled')	Active, Expired or Cancelled

8.5 PAYMENT

Fee payments made against a membership.

Column	Data type	Constraints	Meaning
payment_id	INT(11)	PRIMARY KEY, AUTO_INCREMENT	Payment number
membership_id	INT(11)	FOREIGN KEY → membership, ON DELETE CASCADE; NOT NULL	Membership number
amount	DECIMAL(8,2)	NOT NULL; CHECK amount > 0	Amount paid in rupees
payment_date	DATE	NOT NULL	Date of payment
payment_mode	VARCHAR(15)	NOT NULL; CHECK in ('Cash','Card','UPI','Net Banking')	How the fee was paid

8.6 FITNESS_SESSION

Scheduled group classes, each led by one trainer.

Column	Data type	Constraints	Meaning
session_id	INT(11)	PRIMARY KEY, AUTO_INCREMENT	Session number
session_name	VARCHAR(50)	NOT NULL	Name of the class
session_type	VARCHAR(20)	NOT NULL; CHECK in ('Yoga','Strength','Zumba','CrossFit','Cardio','Pilates')	Category of the class
trainer_id	INT(11)	FOREIGN KEY → trainer, ON DELETE RESTRICT; NOT NULL	Trainer number
session_date	DATE	NOT NULL	Date of the class
start_time	TIME	NOT NULL	Start time
duration_min	INT(11)	NOT NULL; CHECK between 15 and 180	Length in minutes
capacity	INT(11)	NOT NULL; CHECK capacity > 0	Maximum seats
room	VARCHAR(30)	NOT NULL	Room or zone used

Table-level constraint: UNIQUE (room, session_date, start_time), so one room cannot hold two sessions at the same time.

8.7 BOOKING

Seat bookings; resolves the M:N relationship between member and fitness_session.

Column	Data type	Constraints	Meaning
booking_id	INT(11)	PRIMARY KEY, AUTO_INCREMENT	Booking number
member_id	INT(11)	FOREIGN KEY → member, ON DELETE CASCADE; NOT NULL	Member number
session_id	INT(11)	FOREIGN KEY → fitness_session, ON DELETE CASCADE; NOT NULL	Session number
booking_date	DATE	NOT NULL	Date the seat was booked
attendance	VARCHAR(10)	NOT NULL; DEFAULT 'Booked'; CHECK in ('Booked','Attended','Missed','Cancelled')	Booked, Attended, Missed or Cancelled

Table-level constraint: UNIQUE (member_id, session_id), so a member cannot book the same session twice.

9. SQL commands used (DDL, DML) with sample outputs

All commands are in the file `Presentation/gym_management.sql`. Running the file creates the database, the tables and the data, and then runs every query.

9.1 DDL: database creation

```
DROP DATABASE IF EXISTS gym_management;
CREATE DATABASE gym_management;
USE gym_management;
```

9.2 DDL: table creation

```
-- 2.1 MEMBERSHIP_PLAN : the plans the gym sells
CREATE TABLE membership_plan (
    plan_id          INT            AUTO_INCREMENT,
    plan_name        VARCHAR(50)    NOT NULL,
    duration_months  INT            NOT NULL,
    price            DECIMAL(8,2)   NOT NULL,
    description      VARCHAR(150),
    CONSTRAINT pk_plan          PRIMARY KEY (plan_id),
    CONSTRAINT uq_plan_name     UNIQUE (plan_name),
    CONSTRAINT chk_plan_months  CHECK (duration_months BETWEEN 1 AND 24),
    CONSTRAINT chk_plan_price   CHECK (price > 0)
);

-- 2.2 MEMBER : people registered at the gym
CREATE TABLE member (
    member_id   INT           AUTO_INCREMENT,
    first_name  VARCHAR(30)   NOT NULL,
    last_name   VARCHAR(30)   NOT NULL,
    gender      CHAR(1)       NOT NULL,
    dob         DATE          NOT NULL,
    phone       CHAR(10)      NOT NULL,
    email       VARCHAR(60)   NOT NULL,
    city        VARCHAR(40)   NOT NULL DEFAULT 'Hyderabad',
    join_date   DATE          NOT NULL,
    CONSTRAINT pk_member         PRIMARY KEY (member_id),
    CONSTRAINT uq_member_phone   UNIQUE (phone),
    CONSTRAINT uq_member_email   UNIQUE (email),
    CONSTRAINT chk_member_gender CHECK (gender IN ('M', 'F', 'O'))
);

-- 2.3 TRAINER : staff who conduct fitness sessions
CREATE TABLE trainer (
    trainer_id        INT           AUTO_INCREMENT,
    trainer_name      VARCHAR(60)   NOT NULL,
    specialization    VARCHAR(30)   NOT NULL,
    phone             CHAR(10)      NOT NULL,
    email             VARCHAR(60)   NOT NULL,
    experience_years  INT           NOT NULL DEFAULT 0,
    CONSTRAINT pk_trainer        PRIMARY KEY (trainer_id),
    CONSTRAINT uq_trainer_phone  UNIQUE (phone),
    CONSTRAINT uq_trainer_email  UNIQUE (email),
    CONSTRAINT chk_trainer_exp   CHECK (experience_years >= 0)
);

-- 2.4 MEMBERSHIP : one row each time a member buys / renews a plan
CREATE TABLE membership (
    membership_id  INT          AUTO_INCREMENT,
    member_id      INT          NOT NULL,
    plan_id        INT          NOT NULL,
    start_date     DATE         NOT NULL,
    end_date       DATE         NOT NULL,
    status         VARCHAR(10)  NOT NULL DEFAULT 'Active',
    CONSTRAINT pk_membership         PRIMARY KEY (membership_id),
    CONSTRAINT fk_membership_member  FOREIGN KEY (member_id)
        REFERENCES member (member_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_membership_plan    FOREIGN KEY (plan_id)
```

```
        REFERENCES membership_plan (plan_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_membership_dates  CHECK (end_date > start_date),
    CONSTRAINT chk_membership_status CHECK (status IN ('Active', 'Expired', 'Cancelled'))
);

-- 2.5 PAYMENT : fee payments made against a membership
CREATE TABLE payment (
    payment_id     INT            AUTO_INCREMENT,
    membership_id  INT            NOT NULL,
    amount         DECIMAL(8,2)   NOT NULL,
    payment_date   DATE           NOT NULL,
    payment_mode   VARCHAR(15)    NOT NULL,
    CONSTRAINT pk_payment             PRIMARY KEY (payment_id),
    CONSTRAINT fk_payment_membership  FOREIGN KEY (membership_id)
        REFERENCES membership (membership_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT chk_payment_amount     CHECK (amount > 0),
    CONSTRAINT chk_payment_mode       CHECK (payment_mode IN ('Cash', 'Card', 'UPI', 'Net Banking'))
);

-- 2.6 FITNESS_SESSION : scheduled group classes, each led by one trainer
CREATE TABLE fitness_session (
    session_id    INT           AUTO_INCREMENT,
    session_name  VARCHAR(50)   NOT NULL,
    session_type  VARCHAR(20)   NOT NULL,
    trainer_id    INT           NOT NULL,
    session_date  DATE          NOT NULL,
    start_time    TIME          NOT NULL,
    duration_min  INT           NOT NULL,
    capacity      INT           NOT NULL,
    room          VARCHAR(30)   NOT NULL,
    CONSTRAINT pk_session           PRIMARY KEY (session_id),
    CONSTRAINT fk_session_trainer   FOREIGN KEY (trainer_id)
        REFERENCES trainer (trainer_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT uq_session_slot      UNIQUE (room, session_date, start_time),
    CONSTRAINT chk_session_type     CHECK (session_type IN
        ('Yoga', 'Strength', 'Zumba', 'CrossFit', 'Cardio', 'Pilates')),
    CONSTRAINT chk_session_duration CHECK (duration_min BETWEEN 15 AND 180),
    CONSTRAINT chk_session_capacity CHECK (capacity > 0)
);

-- 2.7 BOOKING : resolves the M:N relationship MEMBER <-> FITNESS_SESSION
CREATE TABLE booking (
    booking_id    INT          AUTO_INCREMENT,
    member_id     INT          NOT NULL,
    session_id    INT          NOT NULL,
    booking_date  DATE         NOT NULL,
    attendance    VARCHAR(10)  NOT NULL DEFAULT 'Booked',
    CONSTRAINT pk_booking          PRIMARY KEY (booking_id),
    CONSTRAINT fk_booking_member   FOREIGN KEY (member_id)
        REFERENCES member (member_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_booking_session  FOREIGN KEY (session_id)
        REFERENCES fitness_session (session_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT uq_booking          UNIQUE (member_id, session_id),
    CONSTRAINT chk_booking_att     CHECK (attendance IN ('Booked', 'Attended', 'Missed', 'Cancelled'))
);
```

SHOW TABLES;

Tables_in_gym_management
booking
fitness_session
member
membership
membership_plan
payment
trainer

v_active_members

8 rows in set

DESCRIBE booking;

Field	Type	Null	Key	Default	Extra
booking_id	int(11)	NO	PRI	NULL	auto_increment
member_id	int(11)	NO	MUL	NULL	
session_id	int(11)	NO	MUL	NULL	
booking_date	date	NO		NULL	
attendance	varchar(10)	NO		Booked	

5 rows in set

9.3 DML: INSERT

```
INSERT INTO membership_plan (plan_name, duration_months, price, description) VALUES
('Monthly Basic',       1,  1500.00, 'Gym floor access for one month'),
('Quarterly Standard',  3,  4000.00, 'Gym floor plus group sessions for three months'),
('Half-Yearly Plus',    6,  7500.00, 'Gym floor, group sessions and one diet consultation'),
('Annual Premium',     12, 13000.00, 'All access for a year with locker'),
('Student Monthly',     1,  1000.00, 'Discounted monthly plan for students with ID'),
('Annual Elite',       12, 18000.00, 'Annual Premium plus personal training');

INSERT INTO member (first_name, last_name, gender, dob, phone, email, city, join_date) VALUES
('Arjun',   'Reddy',  'M', '2001-03-14', '9876543210', 'arjun.reddy@example.com',   'Hyderabad',    '2026-06-01'),
('Sneha',   'Iyer',   'F', '1999-11-02', '9876543211', 'sneha.iyer@example.com',    'Secunderabad', '2026-07-10'),
('Rahul',   'Verma',  'M', '1995-07-21', '9876543212', 'rahul.verma@example.com',   'Hyderabad',    '2026-08-05'),
('Priya',   'Sharma', 'F', '2003-01-30', '9876543213', 'priya.sharma@example.com',  'Hyderabad',    '2026-09-15'),
('Karthik', 'Naidu',  'M', '1998-09-09', '9876543214', 'karthik.naidu@example.com', 'Sangareddy',   '2026-05-01'),
('Ananya',  'Gupta',  'F', '2002-05-17', '9876543215', 'ananya.gupta@example.com',  'Hyderabad',    '2026-08-20'),
('Vikram',  'Singh',  'M', '1990-12-05', '9876543216', 'vikram.singh@example.com',  'Secunderabad', '2026-01-10'),
('Meera',   'Joshi',  'F', '2004-08-25', '9876543217', 'meera.joshi@example.com',   'Hyderabad',    '2026-10-01');

INSERT INTO trainer (trainer_name, specialization, phone, email, experience_years) VALUES
('Rohit Malhotra', 'Strength', '9123456701', 'rohit.m@fitzone.example',   8),
('Divya Menon',    'Yoga',     '9123456702', 'divya.m@fitzone.example',   6),
('Imran Khan',     'CrossFit', '9123456703', 'imran.k@fitzone.example',   5),
('Lakshmi Rao',    'Zumba',    '9123456704', 'lakshmi.r@fitzone.example', 4),
('Suresh Kumar',   'Cardio',   '9123456705', 'suresh.k@fitzone.example', 10),
('Neha Kapoor',    'Pilates',  '9123456706', 'neha.k@fitzone.example',    3);

INSERT INTO membership (member_id, plan_id, start_date, end_date, status) VALUES
(1, 4, '2026-06-01', '2027-06-01', 'Active'),
(2, 2, '2026-07-10', '2026-10-10', 'Active'),
(3, 1, '2026-08-05', '2026-09-05', 'Expired'),
(3, 3, '2026-09-06', '2027-03-06', 'Active'),
(4, 5, '2026-09-15', '2026-10-15', 'Active'),
(5, 2, '2026-05-01', '2026-08-01', 'Expired'),
(6, 3, '2026-08-20', '2027-02-20', 'Active'),
(7, 4, '2026-01-10', '2027-01-10', 'Active');

INSERT INTO payment (membership_id, amount, payment_date, payment_mode) VALUES
(1, 13000.00, '2026-06-01', 'UPI'),
(2,  4000.00, '2026-07-10', 'Card'),
(3,  1500.00, '2026-08-05', 'Cash'),
(4,  4000.00, '2026-09-06', 'UPI'),
(4,  2000.00, '2026-09-20', 'UPI'),
(5,  1000.00, '2026-09-15', 'Cash'),
(6,  4000.00, '2026-05-01', 'Card'),
(7,  7500.00, '2026-08-20', 'Net Banking'),
(8, 13000.00, '2026-01-10', 'Card');

INSERT INTO fitness_session
    (session_name, session_type, trainer_id, session_date, start_time, duration_min, capacity, room) VALUES
('Morning Yoga Flow', 'Yoga',     2, '2026-10-05', '06:30:00', 60, 20, 'Studio A'),
('Strength Basics',   'Strength', 1, '2026-10-05', '18:00:00', 75, 12, 'Weights Floor'),
('Zumba Blast',       'Zumba',    4, '2026-10-06', '19:00:00', 45, 25, 'Studio B'),
('CrossFit WOD',      'CrossFit', 3, '2026-10-07', '07:00:00', 60, 15, 'Functional Zone'),
('HIIT Cardio Burn',  'Cardio',   5, '2026-10-08', '18:30:00', 40, 18, 'Cardio Zone'),
('Power Yoga',        'Yoga',     2, '2026-10-09', '07:00:00', 60, 20, 'Studio A'),
('Advanced Lifting',  'Strength', 1, '2026-10-10', '19:00:00', 90, 10, 'Weights Floor');

INSERT INTO booking (member_id, session_id, booking_date, attendance) VALUES
(1, 1, '2026-10-03', 'Attended'),
(2, 1, '2026-10-03', 'Attended'),
(4, 1, '2026-10-04', 'Missed'),
```

```
(6, 1, '2026-10-04', 'Attended'),
(1, 2, '2026-10-03', 'Attended'),
(3, 2, '2026-10-04', 'Attended'),
(7, 2, '2026-10-04', 'Missed'),
(2, 3, '2026-10-05', 'Attended'),
(4, 3, '2026-10-05', 'Attended'),
(3, 4, '2026-10-05', 'Booked'),
(7, 4, '2026-10-06', 'Booked'),
(3, 5, '2026-10-06', 'Booked'),
(6, 5, '2026-10-06', 'Booked'),
(1, 7, '2026-10-06', 'Booked'),
(7, 7, '2026-10-06', 'Booked');
```

Number of records in each table

table_name	total_rows
membership_plan	6
member	8
trainer	6
membership	8
payment	9
fitness_session	7
booking	15

7 rows in set

9.4 Contents of each table after insertion

SELECT * FROM membership_plan;

plan_id	plan_name	duration_months	price	description
1	Monthly Basic	1	1500.00	Gym floor access for one month
2	Quarterly Standard	3	4000.00	Gym floor plus group sessions for three months
3	Half-Yearly Plus	6	7500.00	Gym floor, group sessions and one diet consultation
4	Annual Premium	12	13000.00	All access for a year with locker
5	Student Monthly	1	1000.00	Discounted monthly plan for students with ID
6	Annual Elite	12	18000.00	Annual Premium plus personal training

6 rows in set

SELECT * FROM member;

member_id	first_name	last_name	gender	dob	phone	email	city	join_date
1	Arjun	Reddy	M	2001-03-14	9876543210	arjun.reddy@example.com	Hyderabad	2026-06-01
2	Sneha	Iyer	F	1999-11-02	9876543211	sneha.iyer@example.com	Secunderabad	2026-07-10
3	Rahul	Verma	M	1995-07-21	9876543212	rahul.verma@example.com	Hyderabad	2026-08-05
4	Priya	Sharma	F	2003-01-30	9876543213	priya.sharma@example.com	Hyderabad	2026-09-15
5	Karthik	Naidu	M	1998-09-09	9876543214	karthik.naidu@example.com	Sangareddy	2026-05-01
6	Ananya	Gupta	F	2002-05-17	9876543215	ananya.gupta@example.com	Hyderabad	2026-08-20
7	Vikram	Singh	M	1990-12-05	9876543216	vikram.singh@example.com	Secunderabad	2026-01-10
8	Meera	Joshi	F	2004-08-25	9876543217	meera.joshi@example.com	Hyderabad	2026-10-01

8 rows in set

SELECT * FROM trainer;

trainer_id	trainer_name	specialization	phone	email	experience_years
1	Rohit Malhotra	Strength	9123456701	rohit.m@fitzone.example	8
2	Divya Menon	Yoga	9123456702	divya.m@fitzone.example	6
3	Imran Khan	CrossFit	9123456703	imran.k@fitzone.example	5
4	Lakshmi Rao	Zumba	9123456704	lakshmi.r@fitzone.example	4
5	Suresh Kumar	Cardio	9123456705	suresh.k@fitzone.example	10
6	Neha Kapoor	Pilates	9123456706	neha.k@fitzone.example	3

6 rows in set

SELECT * FROM membership;

membership_id	member_id	plan_id	start_date	end_date	status
1	1	4	2026-06-01	2027-06-01	Active
2	2	2	2026-07-10	2026-10-10	Active
3	3	1	2026-08-05	2026-09-05	Expired
4	3	3	2026-09-06	2027-03-06	Active
5	4	5	2026-09-15	2026-10-15	Active
6	5	2	2026-05-01	2026-08-01	Expired
7	6	3	2026-08-20	2027-02-20	Active
8	7	4	2026-01-10	2027-01-10	Active

8 rows in set

SELECT * FROM payment;

payment_id	membership_id	amount	payment_date	payment_mode
1	1	13000.00	2026-06-01	UPI
2	2	4000.00	2026-07-10	Card
3	3	1500.00	2026-08-05	Cash
4	4	4000.00	2026-09-06	UPI
5	4	2000.00	2026-09-20	UPI
6	5	1000.00	2026-09-15	Cash
7	6	4000.00	2026-05-01	Card
8	7	7500.00	2026-08-20	Net Banking
9	8	13000.00	2026-01-10	Card

9 rows in set

SELECT * FROM fitness_session;

session_id	session_name	session_type	trainer_id	session_date	start_time	duration_min	capacity	room
1	Morning Yoga Flow	Yoga	2	2026-10-05	6:30:00	60	20	Studio A
2	Strength Basics	Strength	1	2026-10-05	18:00:00	75	12	Weights Floor
3	Zumba Blast	Zumba	4	2026-10-06	19:00:00	45	25	Studio B
4	CrossFit WOD	CrossFit	3	2026-10-07	7:00:00	60	15	Functional Zone
5	HIIT Cardio Burn	Cardio	5	2026-10-08	18:30:00	40	18	Cardio Zone
6	Power Yoga	Yoga	2	2026-10-09	7:00:00	60	20	Studio A
7	Advanced Lifting	Strength	1	2026-10-10	19:00:00	90	10	Weights Floor

7 rows in set

SELECT * FROM booking;

booking_id	member_id	session_id	booking_date	attendance
1	1	1	2026-10-03	Attended
2	2	1	2026-10-03	Attended
3	4	1	2026-10-04	Missed
4	6	1	2026-10-04	Attended
5	1	2	2026-10-03	Attended
6	3	2	2026-10-04	Attended
7	7	2	2026-10-04	Missed
8	2	3	2026-10-05	Attended
9	4	3	2026-10-05	Attended
10	3	4	2026-10-05	Booked
11	7	4	2026-10-06	Booked
12	3	5	2026-10-06	Booked
13	6	5	2026-10-06	Booked
14	1	7	2026-10-06	Booked
15	7	7	2026-10-06	Booked

15 rows in set

9.5 DDL: view

```
CREATE OR REPLACE VIEW v_active_members AS
SELECT m.member_id,
       CONCAT(m.first_name, ' ', m.last_name) AS member_name,
       m.phone, p.plan_name, ms.end_date
FROM   member m
JOIN   membership ms     ON ms.member_id = m.member_id AND ms.status = 'Active'
JOIN   membership_plan p ON p.plan_id = ms.plan_id;
```

SELECT * FROM v_active_members ORDER BY end_date;

member_id	member_name	phone	plan_name	end_date
2	Sneha Iyer	9876543211	Quarterly Standard	2026-10-10
4	Priya Sharma	9876543213	Student Monthly	2026-10-15
7	Vikram Singh	9876543216	Annual Premium	2027-01-10
6	Ananya Gupta	9876543215	Half-Yearly Plus	2027-02-20

3	Rahul Verma	9876543212	Half-Yearly Plus	2027-03-06
1	Arjun Reddy	9876543210	Annual Premium	2027-06-01

6 rows in set

9.6 DML: UPDATE and DELETE

A temporary member is inserted, updated and deleted. The DELETE also removes that member's booking through ON DELETE CASCADE, so the counts return to 8 members and 15 bookings.

```
-- 6.1 Insert a temporary member and give them a booking
INSERT INTO member (first_name, last_name, gender, dob, phone, email, city, join_date)
VALUES ('Demo', 'Member', 'M', '2000-01-01', '9000000000', 'demo.member@example.com', 'Hyderabad', '2026-10-06');

INSERT INTO booking (member_id, session_id, booking_date)
SELECT member_id, 6, '2026-10-06' FROM member WHERE phone = '9000000000';

SELECT * FROM member  WHERE phone = '9000000000';
SELECT b.* FROM booking b JOIN member m ON m.member_id = b.member_id WHERE m.phone = '9000000000';

-- 6.2 UPDATE the record
UPDATE member SET city = 'Secunderabad' WHERE phone = '9000000000';
SELECT member_id, first_name, city FROM member WHERE phone = '9000000000';

-- 6.3 DELETE the member; ON DELETE CASCADE removes the booking as well
DELETE FROM member WHERE phone = '9000000000';
SELECT COUNT(*) AS members_left  FROM member;
SELECT COUNT(*) AS bookings_left FROM booking;
```

Statement	Result
INSERT temporary member and booking	member_id 9 and booking_id 16 created
UPDATE member SET city = 'Secunderabad' ...	1 row changed; city shows Secunderabad
DELETE FROM member WHERE phone = '9000000000'	1 row deleted from member; 1 row removed from booking by cascade
SELECT COUNT(*) FROM member / booking	8 members, 15 bookings

10. Queries with outputs

The outputs below were produced by running each query on the sample data.

Q1. All members

```
SELECT * FROM member;
```

member_id	first_name	last_name	gender	dob	phone	email	city	join_date
1	Arjun	Reddy	M	2001-03-14	9876543210	arjun.reddy@example.com	Hyderabad	2026-06-01
2	Sneha	Iyer	F	1999-11-02	9876543211	sneha.iyer@example.com	Secunderabad	2026-07-10
3	Rahul	Verma	M	1995-07-21	9876543212	rahul.verma@example.com	Hyderabad	2026-08-05
4	Priya	Sharma	F	2003-01-30	9876543213	priya.sharma@example.com	Hyderabad	2026-09-15
5	Karthik	Naidu	M	1998-09-09	9876543214	karthik.naidu@example.com	Sangareddy	2026-05-01
6	Ananya	Gupta	F	2002-05-17	9876543215	ananya.gupta@example.com	Hyderabad	2026-08-20
7	Vikram	Singh	M	1990-12-05	9876543216	vikram.singh@example.com	Secunderabad	2026-01-10
8	Meera	Joshi	F	2004-08-25	9876543217	meera.joshi@example.com	Hyderabad	2026-10-01

8 rows in set

Q2. Members from Hyderabad who joined on or after 1 August 2026, newest first

```
SELECT member_id, first_name, last_name, city, join_date
FROM   member
WHERE  city = 'Hyderabad' AND join_date >= '2026-08-01'
ORDER  BY join_date DESC;
```

member_id	first_name	last_name	city	join_date
8	Meera	Joshi	Hyderabad	2026-10-01
4	Priya	Sharma	Hyderabad	2026-09-15
6	Ananya	Gupta	Hyderabad	2026-08-20
3	Rahul	Verma	Hyderabad	2026-08-05

4 rows in set

Q3. INNER JOIN (3 tables): every membership with member and plan details

```
SELECT ms.membership_id,
       CONCAT(m.first_name, ' ', m.last_name) AS member_name,
       p.plan_name, ms.start_date, ms.end_date, ms.status
FROM   membership ms
JOIN   member m          ON m.member_id = ms.member_id
JOIN   membership_plan p ON p.plan_id   = ms.plan_id
ORDER  BY ms.membership_id;
```

membership_id	member_name	plan_name	start_date	end_date	status
1	Arjun Reddy	Annual Premium	2026-06-01	2027-06-01	Active
2	Sneha Iyer	Quarterly Standard	2026-07-10	2026-10-10	Active
3	Rahul Verma	Monthly Basic	2026-08-05	2026-09-05	Expired
4	Rahul Verma	Half-Yearly Plus	2026-09-06	2027-03-06	Active
5	Priya Sharma	Student Monthly	2026-09-15	2026-10-15	Active
6	Karthik Naidu	Quarterly Standard	2026-05-01	2026-08-01	Expired
7	Ananya Gupta	Half-Yearly Plus	2026-08-20	2027-02-20	Active
8	Vikram Singh	Annual Premium	2026-01-10	2027-01-10	Active

8 rows in set

Q4. INNER JOIN: session timetable with the trainer who conducts it

```
SELECT s.session_id, s.session_name, s.session_type, t.trainer_name,
       s.session_date, s.start_time, s.room
FROM   fitness_session s
JOIN   trainer t ON t.trainer_id = s.trainer_id
ORDER  BY s.session_date, s.start_time;
```

session_id	session_name	session_type	trainer_name	session_date	start_time	room
1	Morning Yoga Flow	Yoga	Divya Menon	2026-10-05	6:30:00	Studio A
2	Strength Basics	Strength	Rohit Malhotra	2026-10-05	18:00:00	Weights Floor
3	Zumba Blast	Zumba	Lakshmi Rao	2026-10-06	19:00:00	Studio B
4	CrossFit WOD	CrossFit	Imran Khan	2026-10-07	7:00:00	Functional Zone
5	HIIT Cardio Burn	Cardio	Suresh Kumar	2026-10-08	18:30:00	Cardio Zone
6	Power Yoga	Yoga	Divya Menon	2026-10-09	7:00:00	Studio A
7	Advanced Lifting	Strength	Rohit Malhotra	2026-10-10	19:00:00	Weights Floor

7 rows in set

Q5. JOIN (4 tables): who has booked which session, and with which trainer

```
SELECT b.booking_id,
       CONCAT(m.first_name, ' ', m.last_name) AS member_name,
       s.session_name, t.trainer_name, s.session_date, b.attendance
FROM   booking b
JOIN   member m          ON m.member_id  = b.member_id
JOIN   fitness_session s ON s.session_id = b.session_id
JOIN   trainer t         ON t.trainer_id = s.trainer_id
ORDER  BY s.session_date, member_name;
```

booking_id	member_name	session_name	trainer_name	session_date	attendance
4	Ananya Gupta	Morning Yoga Flow	Divya Menon	2026-10-05	Attended
1	Arjun Reddy	Morning Yoga Flow	Divya Menon	2026-10-05	Attended
5	Arjun Reddy	Strength Basics	Rohit Malhotra	2026-10-05	Attended
3	Priya Sharma	Morning Yoga Flow	Divya Menon	2026-10-05	Missed
6	Rahul Verma	Strength Basics	Rohit Malhotra	2026-10-05	Attended
2	Sneha Iyer	Morning Yoga Flow	Divya Menon	2026-10-05	Attended
7	Vikram Singh	Strength Basics	Rohit Malhotra	2026-10-05	Missed
9	Priya Sharma	Zumba Blast	Lakshmi Rao	2026-10-06	Attended
8	Sneha Iyer	Zumba Blast	Lakshmi Rao	2026-10-06	Attended
10	Rahul Verma	CrossFit WOD	Imran Khan	2026-10-07	Booked
11	Vikram Singh	CrossFit WOD	Imran Khan	2026-10-07	Booked
13	Ananya Gupta	HIIT Cardio Burn	Suresh Kumar	2026-10-08	Booked
12	Rahul Verma	HIIT Cardio Burn	Suresh Kumar	2026-10-08	Booked
14	Arjun Reddy	Advanced Lifting	Rohit Malhotra	2026-10-10	Booked
15	Vikram Singh	Advanced Lifting	Rohit Malhotra	2026-10-10	Booked

15 rows in set

Q6. LEFT JOIN + COUNT: seats booked and seats left in every session (sessions with no bookings still appear)

```
SELECT s.session_name, s.capacity,
       COUNT(b.booking_id)              AS seats_booked,
       s.capacity - COUNT(b.booking_id) AS seats_left
FROM   fitness_session s
LEFT   JOIN booking b ON b.session_id = s.session_id
GROUP  BY s.session_id, s.session_name, s.capacity
ORDER  BY seats_booked DESC;
```

session_name	capacity	seats_booked	seats_left
Morning Yoga Flow	20	4	16
Strength Basics	12	3	9
CrossFit WOD	15	2	13
Zumba Blast	25	2	23
Advanced Lifting	10	2	8
HIIT Cardio Burn	18	2	16
Power Yoga	20	0	20

7 rows in set

Q7. LEFT JOIN + SUM: number of subscriptions and revenue for every plan (plans nobody has bought still appear with zero)

```
SELECT p.plan_name,
       COUNT(DISTINCT ms.membership_id) AS times_sold,
       COALESCE(SUM(pay.amount), 0)     AS revenue
FROM   membership_plan p
LEFT   JOIN membership ms ON ms.plan_id = p.plan_id
LEFT   JOIN payment pay   ON pay.membership_id = ms.membership_id
GROUP  BY p.plan_id, p.plan_name
ORDER  BY revenue DESC;
```

plan_name	times_sold	revenue
Annual Premium	2	26000.00
Half-Yearly Plus	2	13500.00
Quarterly Standard	2	8000.00
Monthly Basic	1	1500.00
Student Monthly	1	1000.00
Annual Elite	0	0.00

6 rows in set

Q8. Aggregate functions on payments

```
SELECT COUNT(*)              AS total_payments,
       SUM(amount)           AS total_collected,
       ROUND(AVG(amount), 2) AS average_payment,
       MIN(amount)           AS smallest_payment,
       MAX(amount)           AS largest_payment
FROM   payment;
```

total_payments	total_collected	average_payment	smallest_payment	largest_payment
9	50000.00	5555.56	1000.00	13000.00

1 row in set

Q9. GROUP BY: collection by payment mode

```
SELECT payment_mode, COUNT(*) AS no_of_payments, SUM(amount) AS amount_collected
FROM   payment
GROUP  BY payment_mode
ORDER  BY amount_collected DESC;
```

payment_mode	no_of_payments	amount_collected
Card	3	21000.00
UPI	3	19000.00
Net Banking	1	7500.00
Cash	2	2500.00

4 rows in set

Q10. GROUP BY + HAVING: members who have booked more than two sessions

```
SELECT CONCAT(m.first_name, ' ', m.last_name) AS member_name,
       COUNT(*) AS sessions_booked
FROM   booking b
JOIN   member m ON m.member_id = b.member_id
GROUP  BY m.member_id, member_name
HAVING COUNT(*) > 2
ORDER  BY sessions_booked DESC;
```

member_name	sessions_booked
Arjun Reddy	3
Rahul Verma	3
Vikram Singh	3

3 rows in set

Q11. Trainer workload: sessions conducted and total bookings handled

```
SELECT t.trainer_name, t.specialization,
       COUNT(DISTINCT s.session_id) AS sessions_conducted,
       COUNT(b.booking_id)          AS total_bookings
FROM   trainer t
LEFT   JOIN fitness_session s ON s.trainer_id = t.trainer_id
LEFT   JOIN booking b         ON b.session_id = s.session_id
GROUP  BY t.trainer_id, t.trainer_name, t.specialization
ORDER  BY total_bookings DESC;
```

trainer_name	specialization	sessions_conducted	total_bookings
Rohit Malhotra	Strength	2	5
Divya Menon	Yoga	2	4
Imran Khan	CrossFit	1	2
Suresh Kumar	Cardio	1	2
Lakshmi Rao	Zumba	1	2
Neha Kapoor	Pilates	0	0

6 rows in set

Q12. Subquery (NOT IN): members who have never booked a session

```
SELECT member_id, first_name, last_name
FROM   member
WHERE  member_id NOT IN (SELECT member_id FROM booking);
```

member_id	first_name	last_name
5	Karthik	Naidu
8	Meera	Joshi

2 rows in set

Q13. Subquery with aggregate: plans priced above the average plan price

```
SELECT plan_name, price
FROM   membership_plan
WHERE  price > (SELECT AVG(price) FROM membership_plan);
```

plan_name	price
Annual Premium	13000.00
Annual Elite	18000.00

2 rows in set

Q14. Balance due: plan price compared with what has been paid so far

```
SELECT ms.membership_id,
       CONCAT(m.first_name, ' ', m.last_name) AS member_name,
       p.plan_name, p.price,
       COALESCE(SUM(pay.amount), 0)           AS paid,
       p.price - COALESCE(SUM(pay.amount), 0) AS balance_due
FROM   membership ms
JOIN   member m          ON m.member_id = ms.member_id
JOIN   membership_plan p ON p.plan_id   = ms.plan_id
LEFT   JOIN payment pay  ON pay.membership_id = ms.membership_id
GROUP  BY ms.membership_id, member_name, p.plan_name, p.price
HAVING balance_due > 0;
```

membership_id	member_name	plan_name	price	paid	balance_due
4	Rahul Verma	Half-Yearly Plus	7500.00	6000.00	1500.00

1 row in set

Q15. Attendance percentage per session type (completed bookings only)

```
SELECT s.session_type,
       SUM(b.attendance = 'Attended')                          AS attended,
       SUM(b.attendance = 'Missed')                            AS missed,
       ROUND(100 * SUM(b.attendance = 'Attended') / COUNT(*), 1) AS attendance_pct
FROM   booking b
JOIN   fitness_session s ON s.session_id = b.session_id
WHERE  b.attendance IN ('Attended', 'Missed')
GROUP  BY s.session_type;
```

session_type	attended	missed	attendance_pct
Strength	2	1	66.7
Yoga	3	1	75.0
Zumba	2	0	100.0

3 rows in set

11. UI design and screenshots

The interface is a web application with a fixed menu on the left and one screen per table. Each screen has the same layout: an insert form at the top and the list of records below it with a Delete button on every row. A coloured message at the top reports the result of the last action, including any constraint error returned by MySQL.

Screen	Tables used	Operations
Dashboard	All	View: live counts and session timetable
Members	member, membership, membership_plan	View, insert, delete
Memberships	membership, membership_plan, payment, member	View, insert (membership and payment in one transaction), delete
Trainers	trainer, fitness_session	View, insert, delete
Fitness Sessions	fitness_session, trainer, booking	View, insert, delete
Bookings	booking, member, fitness_session, trainer	View, insert, delete
Reports	All	View: join and aggregate queries

11.1 Dashboard

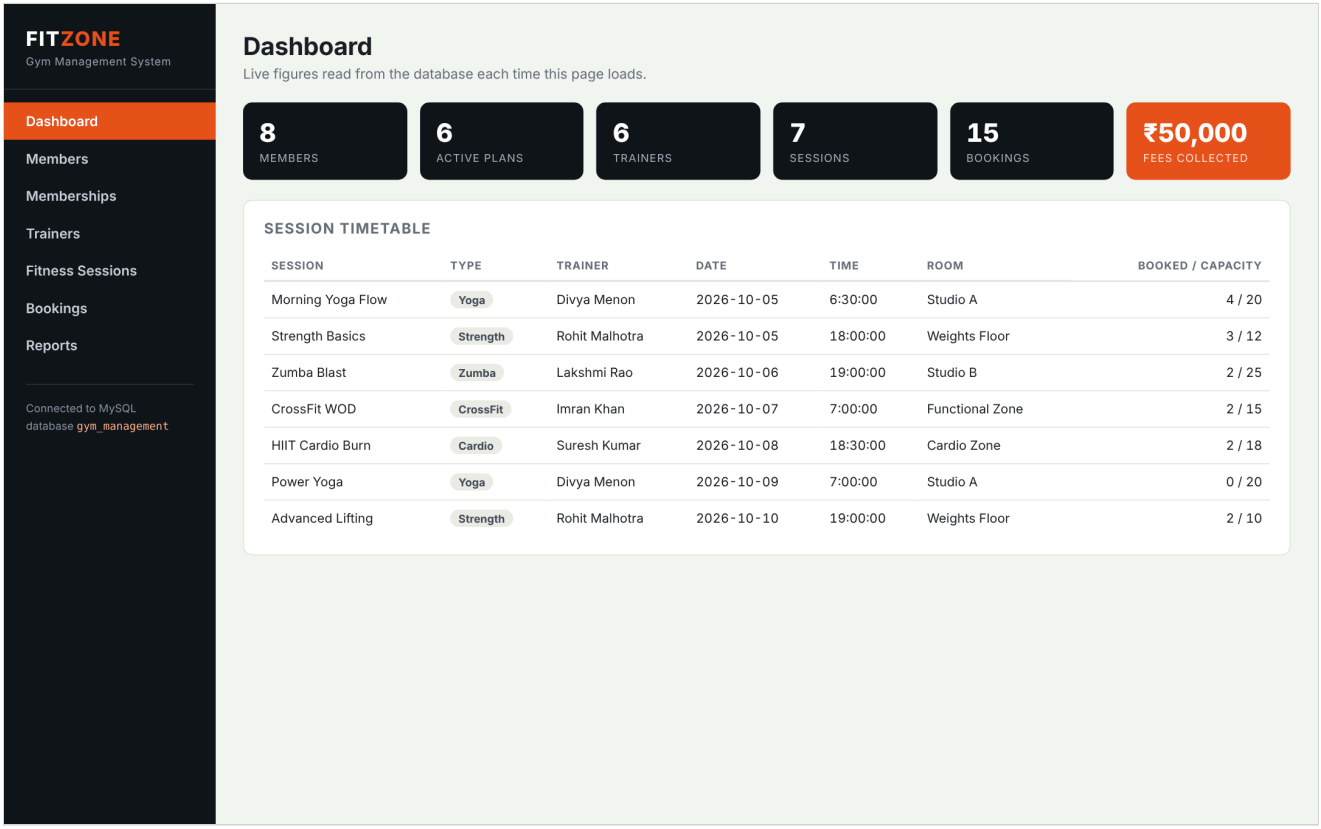


Figure 11.1: Dashboard with live counts

11.2 Insert a member (before and after)

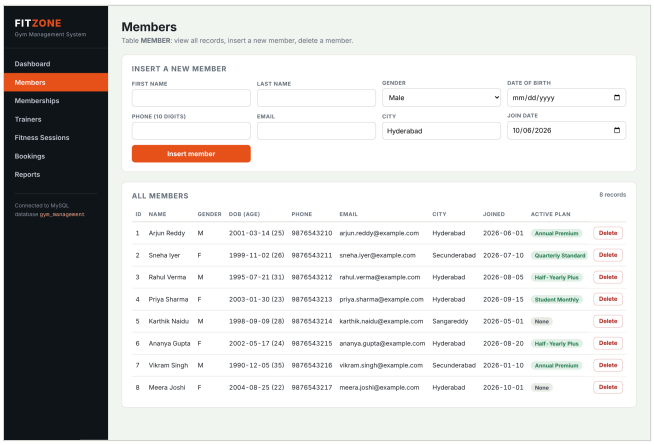


Figure 11.2: Members screen before insert (8 records)

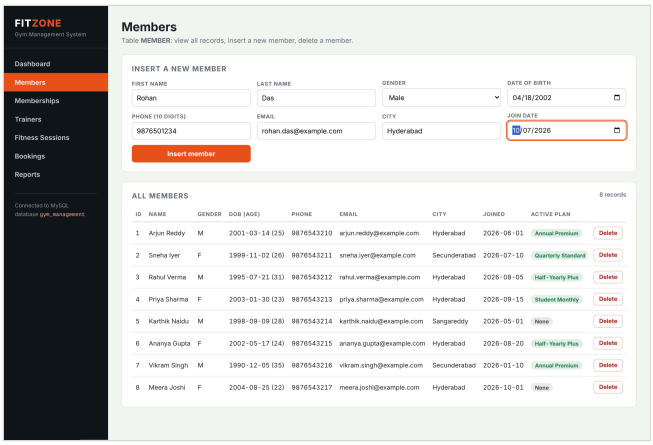


Figure 11.3: Insert form filled in

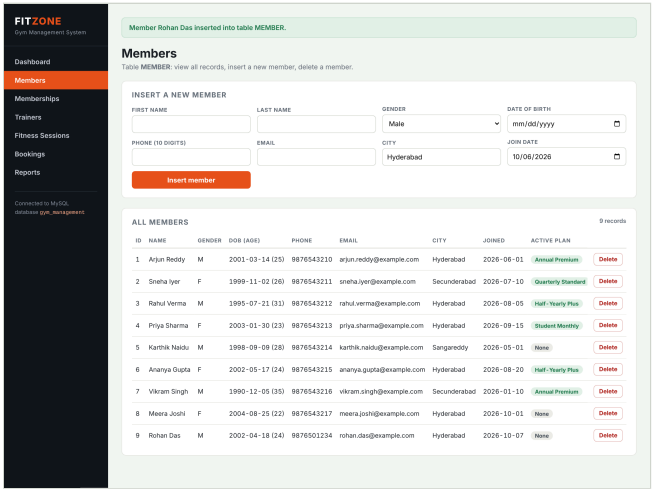
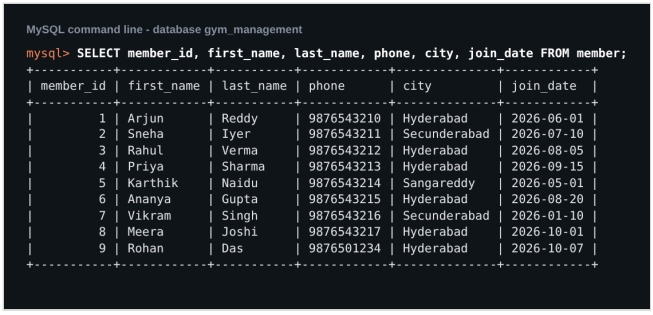


Figure 11.4: After insert (9 records)



MySQL command line - database gym_management

```
mysql> SELECT member_id, first_name, last_name, phone, city, join_date FROM member;
+-----------+------------+-----------+------------+-------------+------------+
| member_id | first_name | last_name | phone      | city        | join_date  |
+-----------+------------+-----------+------------+-------------+------------+
|         1 | Arjun      | Reddy     | 9876543210 | Hyderabad   | 2026-06-01 |
|         2 | Sneha      | Iyer      | 9876543211 | Secunderabad| 2026-07-10 |
|         3 | Rahul      | Verma     | 9876543212 | Hyderabad   | 2026-08-05 |
|         4 | Priya      | Sharma    | 9876543213 | Hyderabad   | 2026-09-15 |
|         5 | Karthik    | Naidu     | 9876543214 | Sangareddy  | 2026-05-01 |
|         6 | Ananya     | Gupta     | 9876543215 | Hyderabad   | 2026-08-20 |
|         7 | Vikram     | Singh     | 9876543216 | Secunderabad| 2026-01-10 |
|         8 | Meera      | Joshi     | 9876543217 | Hyderabad   | 2026-10-01 |
|         9 | Rohan      | Das       | 9876501234 | Hyderabad   | 2026-10-07 |
+-----------+------------+-----------+------------+-------------+------------+
```

Figure 11.5: The new row seen in MySQL

11.3 Insert a session and a booking

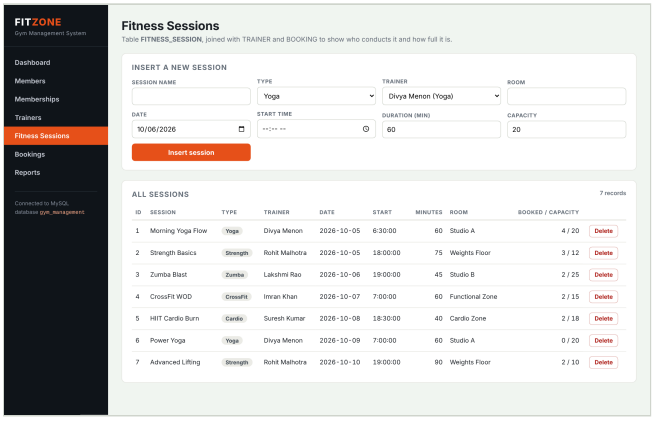


Figure 11.6: Sessions before insert (7 records)

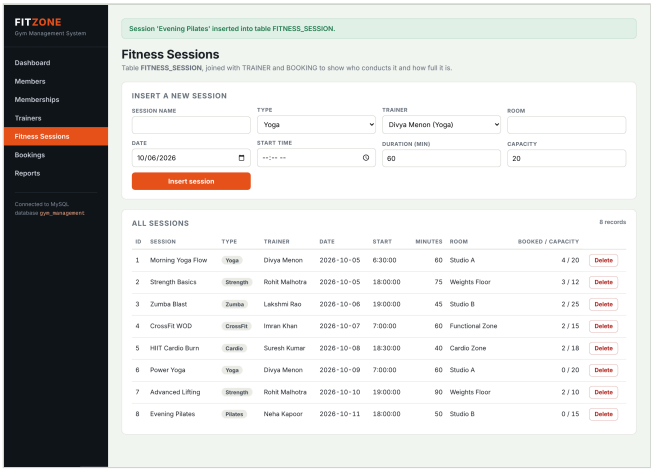


Figure 11.7: Sessions after insert (8 records)

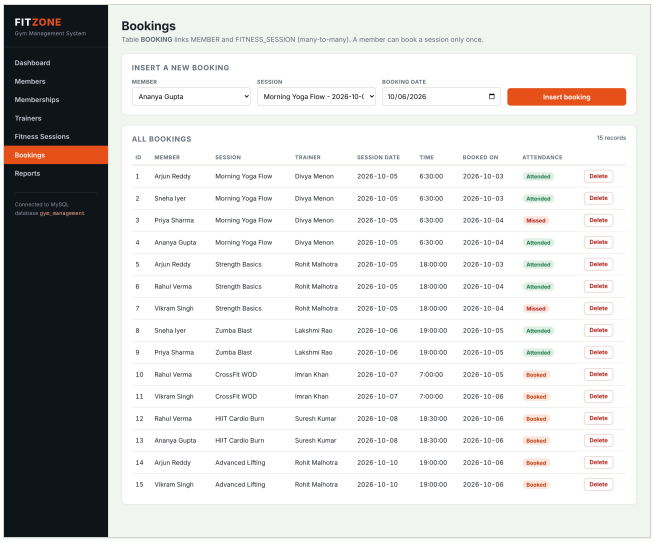


Figure 11.8: Bookings before insert (15 records)

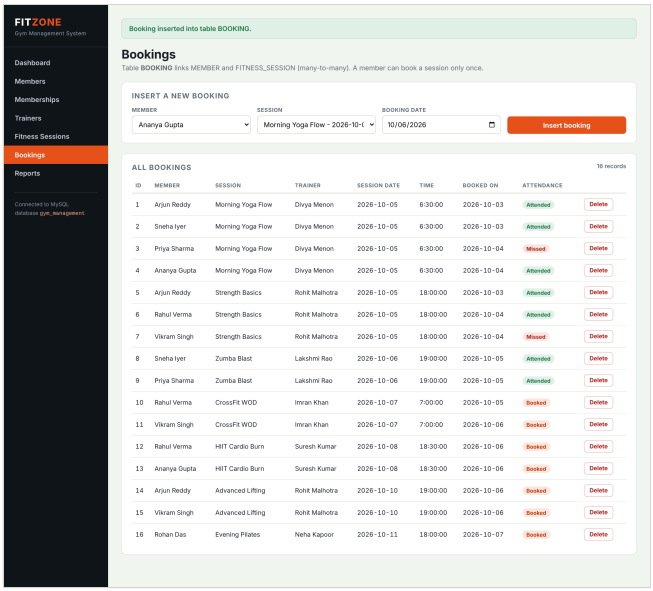


Figure 11.9: Bookings after insert (16 records)

```
MySQL command line - database gym_management
mysql> SELECT session_id, session_name, trainer_id, session_date, room FROM fitness_session WHERE session_id >= 6;
+------------+-----------------+------------+--------------+---------------+
| session_id | session_name    | trainer_id | session_date | room          |
+------------+-----------------+------------+--------------+---------------+
|          6 | Power Yoga      |          2 | 2026-10-09   | Studio A      |
|          7 | Advanced Lifting |          1 | 2026-10-10   | Weights Floor |
|          8 | Evening Pilates |          6 | 2026-10-11   | Studio B      |
+------------+-----------------+------------+--------------+---------------+

mysql> SELECT * FROM booking WHERE booking_id >= 14;
+------------+-----------+------------+--------------+------------+
| booking_id | member_id | session_id | booking_date | attendance |
+------------+-----------+------------+--------------+------------+
|         14 |         1 |          7 | 2026-10-06   | Booked     |
|         15 |         7 |          7 | 2026-10-06   | Booked     |
|         16 |         9 |          8 | 2026-10-07   | Booked     |
+------------+-----------+------------+--------------+------------+
```

Figure 11.10: New session and booking seen in MySQL

11.4 Delete a booking and a session

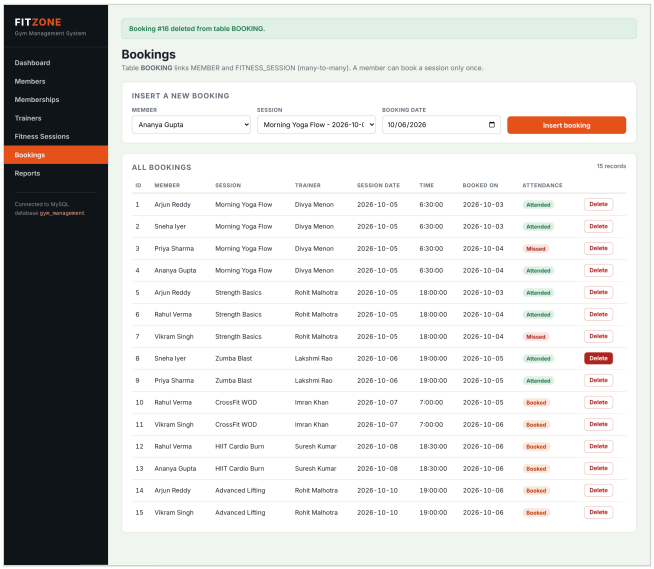


Figure 11.11: Bookings after delete (15 records)

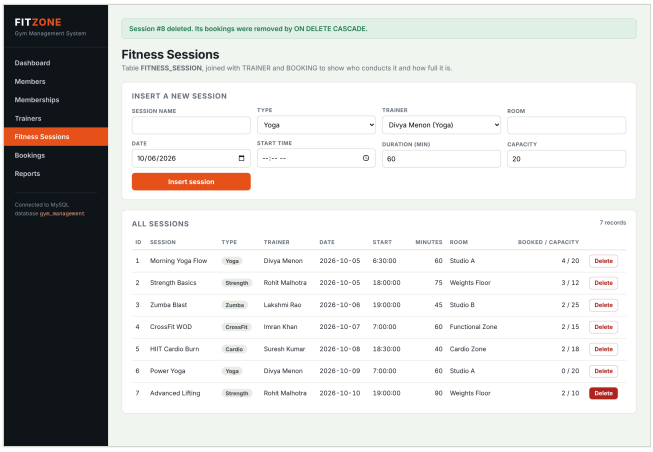


Figure 11.12: Sessions after delete (7 records)

11.5 Memberships: insert as one transaction

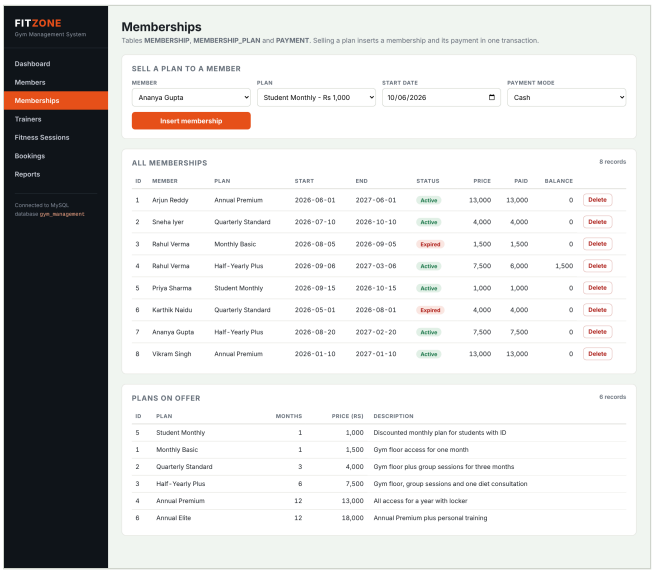


Figure 11.13: Memberships before insert

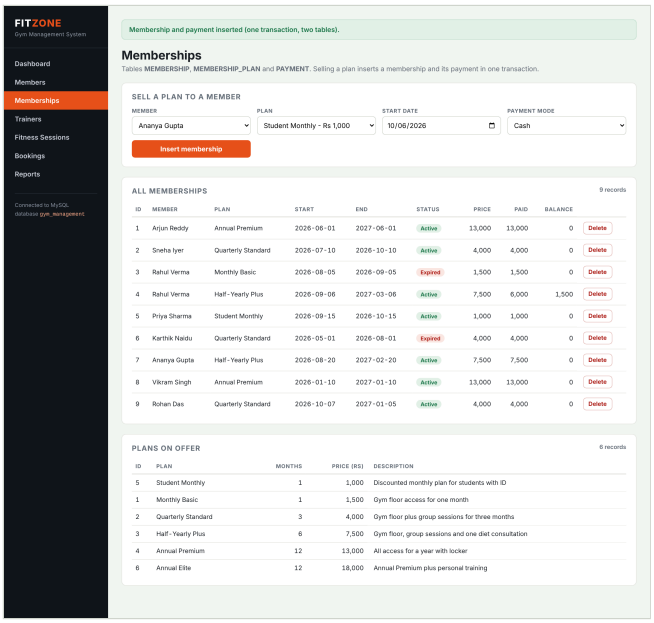


Figure 11.14: After insert: membership 9 with its payment

11.6 Delete a member (before and after, with cascade)

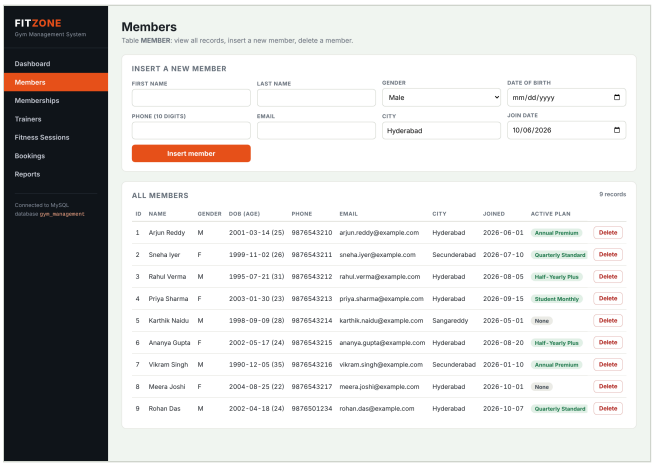


Figure 11.15: Members before delete (9 records)

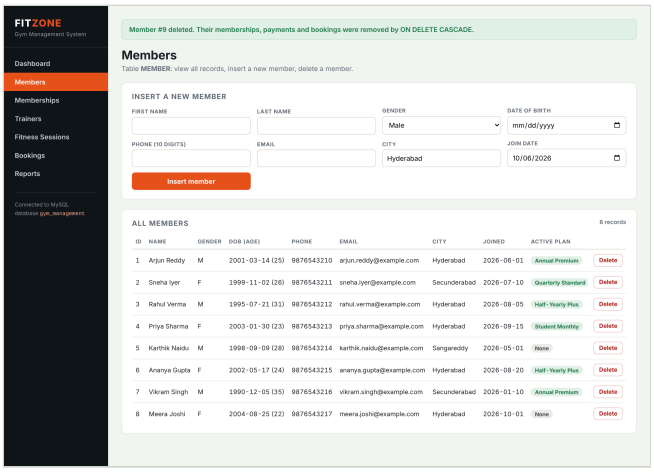


Figure 11.16: Members after delete (8 records)

```
MySQL command line - database gym_management
mysql> SELECT member_id, first_name, last_name FROM member WHERE member_id = 9;
+-----------+------------+-----------+
| member_id | first_name | last_name |
+-----------+------------+-----------+
|         9 | Rohan      | Das       |
+-----------+------------+-----------+

mysql> SELECT * FROM membership WHERE member_id = 9;
+---------------+-----------+---------+------------+------------+--------+
| membership_id | member_id | plan_id | start_date | end_date   | status |
+---------------+-----------+---------+------------+------------+--------+
|             9 |         9 |       2 | 2026-10-07 | 2027-01-05 | Active |
+---------------+-----------+---------+------------+------------+--------+

mysql> SELECT * FROM payment WHERE membership_id = 9;
+------------+---------------+---------+--------------+--------------+
| payment_id | membership_id | amount  | payment_date | payment_mode |
+------------+---------------+---------+--------------+--------------+
|         10 |             9 | 4000.00 | 2026-10-07   | UPI          |
+------------+---------------+---------+--------------+--------------+
```

Figure 11.17: MySQL before delete: member 9 has a membership and a payment

```
MySQL command line - database gym_management
mysql> SELECT member_id, first_name, last_name, phone FROM member;
+-----------+------------+-----------+------------+
| member_id | first_name | last_name | phone      |
+-----------+------------+-----------+------------+
|         1 | Arjun      | Reddy     | 9876543210 |
|         2 | Sneha      | Iyer      | 9876543211 |
|         3 | Rahul      | Verma     | 9876543212 |
|         4 | Priya      | Sharma    | 9876543213 |
|         5 | Karthik    | Naidu     | 9876543214 |
|         6 | Ananya     | Gupta     | 9876543215 |
|         7 | Vikram     | Singh     | 9876543216 |
|         8 | Meera      | Joshi     | 9876543217 |
+-----------+------------+-----------+------------+

mysql> SELECT COUNT(*) AS memberships_of_member_9 FROM membership WHERE member_id = 9;
+------------------------+
| memberships_of_member_9 |
+------------------------+
|                      0 |
+------------------------+

mysql> SELECT COUNT(*) AS payments_of_membership_9 FROM payment WHERE membership_id = 9;
+-------------------------+
| payments_of_membership_9 |
+-------------------------+
|                       0 |
+-------------------------+
```

Figure 11.18: MySQL after delete: all three rows are gone

11.7 Constraint errors shown in the UI

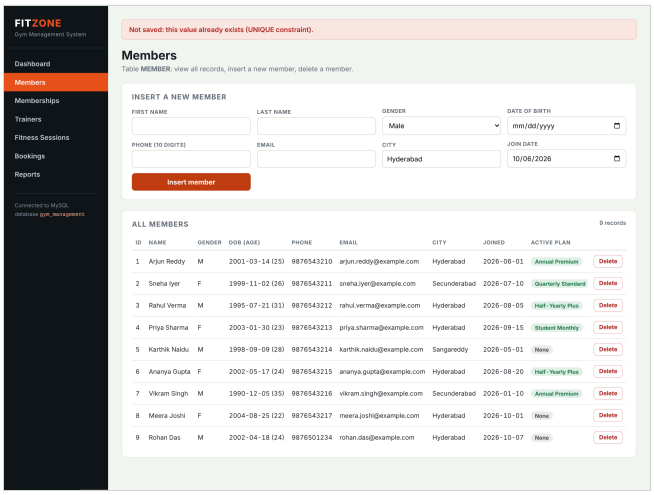


Figure 11.19: Duplicate phone number rejected (UNIQUE)

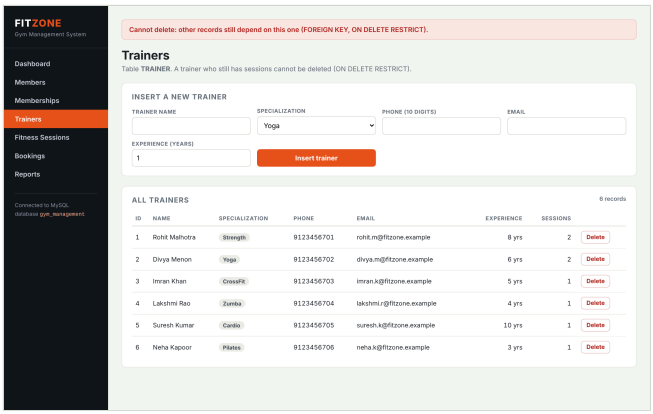


Figure 11.20: Trainer with sessions cannot be deleted (FOREIGN KEY)

11.8 Reports

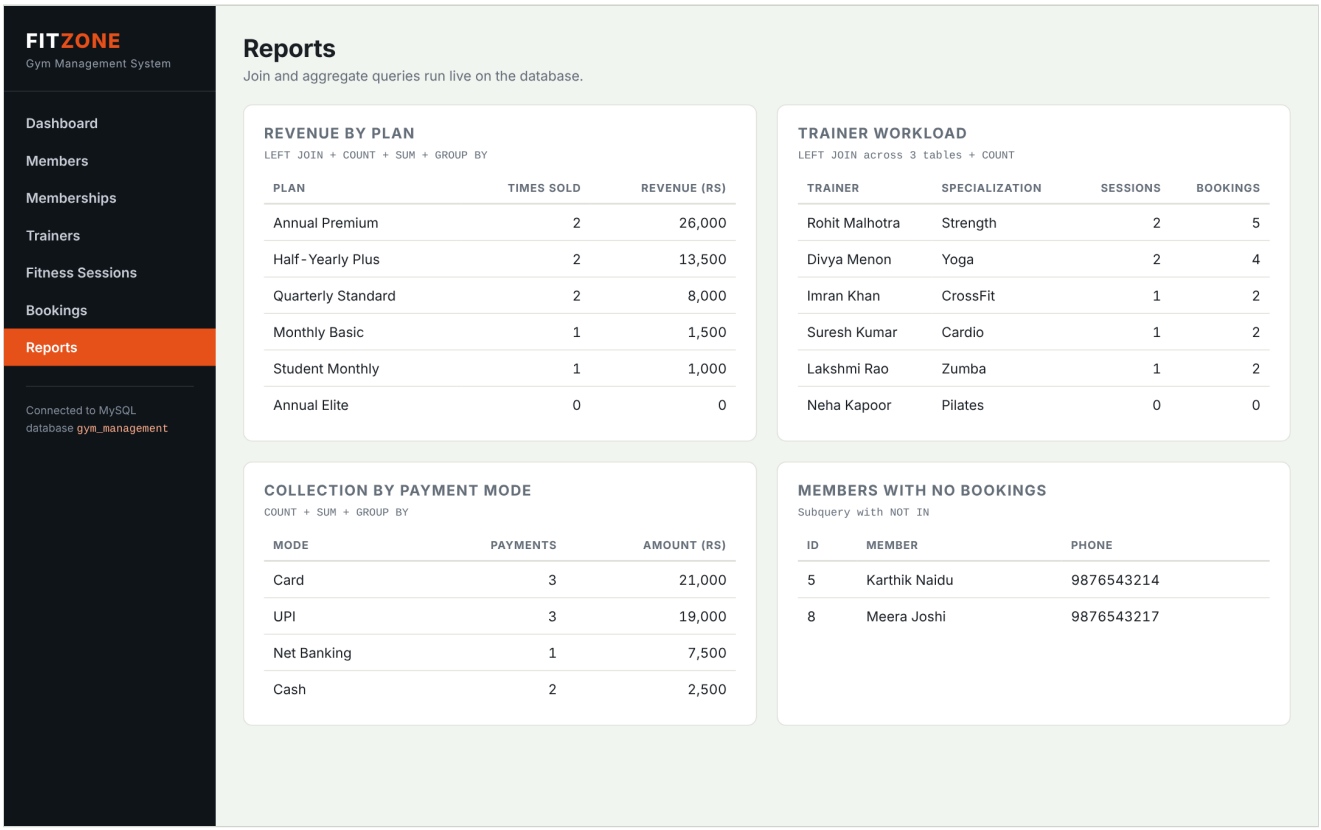


Figure 11.21: Reports screen

12. Implementation details

12.1 Technology stack

Layer	Technology	Role
Database	MySQL 8.0	Stores the seven tables and enforces all constraints
Driver	PyMySQL	Sends SQL from Python to MySQL and returns the rows
Application	Python 3, Flask	Receives requests, runs the SQL, renders the pages
Presentation	HTML, CSS, Jinja2	Forms and tables shown in the browser

Request flow: Browser → Flask route in `app.py` → PyMySQL → MySQL → rows returned → Jinja2 template → HTML page.

12.2 Source files

File	Purpose
app.py	All routes and database functions
config.py	Database host, user, password and database name
templates/base.html	Common page layout and menu
templates/*.html	One template per screen
static/style.css	Styling
requirements.txt	Python packages to install

12.3 Database connectivity

Connection details are read from `config.py`. Each request opens a connection, runs its statement and closes the connection.

```
def get_connection():
    """Open a new connection to the MySQL database."""
    return pymysql.connect(
        host=config.DB_HOST,
        port=config.DB_PORT,
        user=config.DB_USER,
        password=config.DB_PASSWORD,
        database=config.DB_NAME,
        cursorclass=pymysql.cursors.DictCursor,
    )
```

Two helper functions are used by every screen: one for SELECT statements and one for statements that change data. The second commits on success and rolls back on error.

```
def fetch_all(sql, params=()):
    """Run a SELECT statement and return all rows as a list of dictionaries."""
    conn = get_connection()
    try:
        with conn.cursor() as cur:
            cur.execute(sql, params)
            return cur.fetchall()
    finally:
        conn.close()
```

```
def run(sql, params=()):
    """Run an INSERT / UPDATE / DELETE statement and commit it.
    Returns the number of rows affected."""
    conn = get_connection()
    try:
        with conn.cursor() as cur:
            affected = cur.execute(sql, params)
        conn.commit()
        return affected
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()
```

12.4 Insert and delete

Values typed by the user are never joined into the SQL text. They are passed separately as parameters (`%s`), which prevents SQL injection.

```
@app.route("/members/add", methods=["POST"])
def add_member():
    f = request.form
    save(
        """INSERT INTO member (first_name, last_name, gender, dob, phone, email, city, join_date)
           VALUES (%s, %s, %s, %s, %s, %s, %s, %s)""",
        (f["first_name"].strip(), f["last_name"].strip(), f["gender"], f["dob"],
         f["phone"].strip(), f["email"].strip(), f["city"].strip(), f["join_date"]),
        f"Member {f['first_name']} {f['last_name']} inserted into table MEMBER.",
    )
    return redirect(url_for("members"))
```

```
@app.route("/members/delete/<int:member_id>", methods=["POST"])
def delete_member(member_id):
    save("DELETE FROM member WHERE member_id = %s", (member_id,),
         f"Member #{member_id} deleted. Their memberships, payments and bookings "
         "were removed by ON DELETE CASCADE.")
    return redirect(url_for("members"))
```

12.5 View with a join

```
@app.route("/bookings")
def bookings():
    rows = fetch_all(
        """SELECT b.booking_id, CONCAT(m.first_name, ' ', m.last_name) AS member_name,
                  s.session_name, t.trainer_name, s.session_date, s.start_time,
                  b.booking_date, b.attendance
           FROM booking b
           JOIN member m          ON m.member_id  = b.member_id
           JOIN fitness_session s ON s.session_id = b.session_id
           JOIN trainer t         ON t.trainer_id = s.trainer_id
           ORDER BY b.booking_id"""
    )
    member_list = fetch_all("SELECT member_id, first_name, last_name FROM member ORDER BY first_name")
    session_list = fetch_all(
        """SELECT s.session_id, s.session_name, s.session_date, s.capacity, COUNT(b.booking_id) AS booked
           FROM fitness_session s LEFT JOIN booking b ON b.session_id = s.session_id
           GROUP BY s.session_id ORDER BY s.session_date, s.start_time"""
    )
    return render_template("bookings.html", rows=rows, members=member_list,
                           sessions=session_list, today=date.today().isoformat())
```

12.6 Transaction across two tables

Selling a plan must insert a row in MEMBERSHIP and a row in PAYMENT. Both statements run on one connection and are committed together; if either fails, both are rolled back.

```
@app.route("/memberships/add", methods=["POST"])
def add_membership():
    """Insert a membership and its first payment together, as one transaction."""
    f = request.form
    conn = get_connection()
    try:
        with conn.cursor() as cur:
            cur.execute("SELECT duration_months, price FROM membership_plan WHERE plan_id = %s", (f["plan_id"],))
            plan = cur.fetchone()
            start = date.fromisoformat(f["start_date"])
            end = start + timedelta(days=30 * plan["duration_months"])
            cur.execute(
                "INSERT INTO membership (member_id, plan_id, start_date, end_date, status) VALUES (%s, %s, %s, %s, 'Active')",
                (f["member_id"], f["plan_id"], start, end),
            )
            cur.execute(
                "INSERT INTO payment (membership_id, amount, payment_date, payment_mode) VALUES (%s, %s, %s, %s)",
                (cur.lastrowid, plan["price"], start, f["payment_mode"]),
            )
        conn.commit()                    # both rows are saved together
        flash("Membership and payment inserted (one transaction, two tables).", "ok")
    except pymysql.MySQLError as error:
        conn.rollback()                  # neither row is saved
        flash(explain(error), "error")
    finally:
        conn.close()
    return redirect(url_for("memberships"))
```

12.7 Reporting constraint errors

When MySQL rejects a statement, the error number is translated into a message for the user.

```
def explain(error):
    """Turn a MySQL error into a message a user can understand."""
    code = error.args[0] if error.args else 0
    if code == 1062:
        return "Not saved: this value already exists (UNIQUE constraint)."
    if code == 1452:
        return "Not saved: the linked record does not exist (FOREIGN KEY constraint)."
    if code == 1451:
        return "Cannot delete: other records still depend on this one (FOREIGN KEY, ON DELETE RESTRICT)."
    if code in (3819, 4025):
        return "Not saved: a value is outside the allowed range (CHECK constraint)."
    return f"Database error {code}: {error.args[-1]}"
```

12.8 How to run

```
mysql -u root -p < gym_management.sql
cd UI-Source-Code
pip install -r requirements.txt
python app.py
# open http://127.0.0.1:5000
```

13. Testing

The database was tested first from the MySQL prompt and then through the interface. Each test was run on the sample data.

13.1 Database tests

No.	Test case	Expected result	Actual result	Status
D1	Run gym_management.sql from start to end	Database, 7 tables and 59 records created; all queries run	As expected, no errors	Pass
D2	Row count of each table	At least 5 rows in every table	6, 8, 6, 8, 9, 7, 15	Pass
D3	Insert a member with an existing phone number	Rejected by UNIQUE	Error 1062: Duplicate entry '9876543210' for key 'uq_member_phone'	Pass
D4	Insert booking (member 1, session 1) a second time	Rejected by UNIQUE (member_id, session_id)	Error 1062: Duplicate entry '1-1' for key 'uq_booking'	Pass
D5	Insert a booking for member_id 99, which does not exist	Rejected by FOREIGN KEY	Error 1452: a foreign key constraint fails (fk_booking_member)	Pass
D6	Insert a payment with amount -50	Rejected by CHECK	Rejected: constraint chk_payment_amount violated	Pass
D7	Delete trainer 1, who has two sessions	Rejected by ON DELETE RESTRICT	Error 1451: a foreign key constraint fails (fk_session_trainer)	Pass
D8	Delete a member who has a booking	Member deleted; booking removed by CASCADE	Members back to 8, bookings back to 15	Pass
D9	LEFT JOIN query Q6	Session with no bookings is still listed	Power Yoga shown with 0 booked, 20 left	Pass
D10	Aggregate query Q8	Totals match the data	9 payments, total 50000.00, average 5555.56	Pass

13.2 User interface tests

No.	Test case	Expected result	Actual result	Status
U1	Open each of the 7 screens	Page loads and shows current records	All pages load	Pass
U2	Insert member Rohan Das	Record added; count 8 to 9; row visible in MySQL	Added as member 9; seen with SELECT in MySQL	Pass
U3	Insert a member with a phone number already used	Not saved; error shown	Message: value already exists (UNIQUE constraint)	Pass
U4	Insert session Evening Pilates	Record added; count 7 to 8	Added as session 8	Pass
U5	Insert a booking for Rohan Das in Evening Pilates	Record added; count 15 to 16	Added as booking 16; seen in MySQL	Pass

U6	Delete that booking	Record removed; count 16 to 15	Removed	Pass
U7	Delete trainer Rohit Malhotra	Not deleted; error shown	Message: other records still depend on this one	Pass
U8	Delete session Evening Pilates	Record removed; count 8 to 7	Removed	Pass
U9	Sell Quarterly Standard to Rohan Das	One membership and one payment inserted	Membership 9 with payment of 4000 shown as fully paid	Pass
U10	Delete member Rohan Das	Member removed with membership and payment	Count 9 to 8; 0 memberships and 0 payments left for him in MySQL	Pass

14. Conclusion and future enhancements

14.1 Conclusion

The project delivers a working database management system for gym membership and fitness session management. The ER design was mapped to seven tables in third normal form, so each fact is stored once. Keys and constraints in the database prevent duplicate members, bookings for members or sessions that do not exist, negative payments and double booking of a seat. The queries show that the data can answer the questions a gym actually asks, and the Flask interface lets staff insert, delete and view records without writing SQL, while every change is immediately reflected in MySQL.

14.2 Future enhancements

- User login with separate roles for staff, trainers and members.
- A scheduled MySQL event that marks memberships as Expired when the end date passes.
- A trigger that rejects a booking when the session is full, so the rule also holds outside the interface.
- Edit (UPDATE) forms in the interface and attendance marking by trainers.
- Online fee payment and renewal reminders by SMS or email.
- Charts for monthly revenue and attendance trends.

15. References

1. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th edition, Pearson, 2016.
2. A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th edition, McGraw-Hill, 2019.
3. Oracle Corporation, *MySQL 8.0 Reference Manual*, https://dev.mysql.com/doc/refman/8.0/en/
4. Pallets Projects, *Flask Documentation*, https://flask.palletsprojects.com/
5. PyMySQL contributors, *PyMySQL Documentation*, https://pymysql.readthedocs.io/
6. DBMS course lecture notes on ER modelling, the relational model, keys and normalization.

16. Contribution of each member

Edit this table so that it matches what each member actually did and what the GitHub commit history shows.

Member	Roll number	Contribution
Sai Nihal Nellore	25WU0101118	*[for example: Flask user interface, database connectivity, UI testing and screenshots]*
[Member 2 name]	*[Roll number]*	*[for example: requirement analysis, ER diagram, relational schema, normalization, data dictionary]*
[Member 3 name]	*[Roll number]*	*[for example: table creation, sample data, SQL queries, database testing, report and slides]*

17. Appendix: GitHub repository link

Repository: *https://github.com/[username]/DBMS-Course-Project*

```
DBMS-Course-Project/
|-- README.md
|-- Presentation/
|   |-- Gym_Management_Presentation.pptx
|   |-- Gym_Management_Presentation.pdf
|   |-- ER_Diagram.png
|   |-- gym_management.sql
|   |-- UI-Source-Code/
|   |   |-- app.py
|   |   |-- config.py
|   |   |-- requirements.txt
|   |   |-- templates/
|   |   `-- static/
|   `-- Screenshots/          (24 screenshots: view, insert, delete, before and after)
`-- Project-Report/
    `-- Project_Report.pdf
```